

Same-Origin Policy

DR. ADNAN

What should be allowed?

ما الذي يجب السماح به؟

- ▶ Should site A be able to **link to** site B? هل يجب أن يكون الموقع "أ" قادرًا على الارتباط بالموقع "ب"؟
- ▶ Should site A be able to **embed** site B? هل يجب أن يكون الموقع "أ" قادرًا على تضمين الموقع "ب"؟
- ▶ Should site A be able to **embed** site B and **modify** its contents? هل يجب أن يكون الموقع "أ" قادرًا على تضمين الموقع "ب" وتعديل محتوياته؟
- ▶ Should site A be able to **submit a form** to site B? هل يجب أن يكون الموقع "أ" قادرًا على إرسال نموذج إلى الموقع "ب"؟
- ▶ Should site A be able to **embed images** from site B? هل يجب أن يكون الموقع "أ" قادرًا على تضمين الصور من الموقع "ب"؟
- ▶ Should site A be able to **embed scripts** from site B? هل يجب أن يكون الموقع "أ" قادرًا على تضمين البرامج النصية من الموقع "ب"؟
- ▶ Should site A be able to **read data** from site B? هل يجب أن يكون الموقع "أ" قادرًا على قراءة البيانات من الموقع "ب"؟

Same Origin Policy

نفس سياسة المنشأ

- ▶ This is the fundamental security model of the web
- ▶ **If you remember one thing from this class, this is it:**
 - ▶ Two pages from different sources should not be allowed to interfere with each other

- ▶ هذا هو نموذج الأمان الأساسي للويب
- ▶ إذا كنت تتذكر شيئاً واحداً من هذا الفصل، فهو:
 - ▶ لا ينبغي السماح لصفحتين من مصادر مختلفة بالتداخل مع بعضهما البعض

The web is an operating system

الويب هو نظام تشغيل

- ▶ An **origin** is analogous to an OS **process**
- ▶ The **web browser** itself is analogous to an OS **kernel**
- ▶ Sites rely on the browser to enforce all the system's security rules
- ▶ If there's a bug in the browser itself then all these rules go out the window (just like in an OS)

- ▶ الأصل مشابه لعملية نظام التشغيل
- ▶ يشبه متصفح الويب نفسه نواة نظام التشغيل
- ▶ تعتمد المواقع على المتصفح لفرض كافة قواعد أمان النظام
- ▶ إذا كان هناك خطأ في المتصفح نفسه، فستختفي كل هذه القواعد (تمامًا كما هو الحال في نظام التشغيل)

The Basic Rule

القاعدة الأساسية

- ▶ Given **two separate JavaScript execution contexts**, one should be able to access the other only if the **protocols, hostnames, and port numbers** associated with their host documents **match exactly**.
- ▶ This "**protocol-host-port tuple**" is called an "**origin**".

- ▶ بالنظر إلى سياقين منفصلين لتنفيذ JavaScript، يجب أن يكون أحدهما قادرًا على الوصول إلى الآخر فقط إذا كانت بروتوكولات وأسماء المضيفين وأرقام المنافذ المرتبطة بمستندات المضيف تتطابق تمامًا.
- ▶ يُطلق على "مجموعة منفذ مضيف البروتوكول" اسم "الأصل".



<https://example.com:4000/a/b.html?user=Alice&year=2019#p2>

Protocol

Hostname

Port

Path

Query

Fragment

Same origin policy

نفس سياسة المنشأ

```
function isSameOrigin(url1, url2) {  
    return url1.protocol === url2.protocol &&  
        url1.hostname === url2.hostname &&  
        url1.port === url2.port  
}
```

What should be allowed?

ما الذي يجب السماح به؟

- ▶ Where does one **document** begin and another end?
- ▶ How much **interaction** should be allowed between non-cooperating origins?
- ▶ Which **actions** should be subject to security checks?

- ▶ أين يبدأ مستند وينتهي آخر؟
- ▶ ما مقدار التفاعل الذي يجب السماح به بين الأصول غير المتعاونة؟
- ▶ ما هي الإجراءات التي يجب أن تخضع لفحوصات الأمان؟

Demo: Same origin policy

العرض التوضيحي: نفس سياسة المنشأ

From <https://web.stanford.edu/class/cs253/>:

```
const iframe = document.createElement('iframe')
iframe.src = 'https://web.stanford.edu/class/cs106a/'
document.body.appendChild(iframe)

// Same origin, so everything is allowed
iframe.contentDocument.body.style.backgroundColor = 'red'

// Even this!
let i = 0
const imgs = iframe.contentDocument.body.querySelectorAll('img')
setInterval(() => {
  imgs.forEach(img => img.style.transform = `rotate(${i}deg)`)
  i += 1
}, 50)
```

Demo: Same origin policy (pt2)

العرض التوضيحي: نفس سياسة الأصل (pt2)

From <https://web.stanford.edu/class/cs253/>:

```
const iframe = document.createElement('iframe')
iframe.src = 'https://crypto.stanford.edu'
document.body.appendChild(iframe)
```

```
// Is this allowed?
```

```
document.body.appendChild(iframe) // Allowed!
```

```
// Is this allowed?
```

```
iframe.contentDocument.body.style.backgroundColor = 'red' // Not allowed!
```

```
// Navigate to Dan's homepage, then...
```

```
// Is this allowed?
```

```
iframe.src = 'https://example.com' // Allowed! Surprised?
```

Demo: Is cross-origin fetch allowed?

العرض التوضيحي: هل يُسمح بالجلب عبر الأصل ؟

```
//From https://web.stanford.edu/class/cs253/:  
const res = await fetch('https://axess.stanford.edu')  
const data = await res.text()  
console.log(data)
```

- ▶ No! Would be a huge violation of Same Origin Policy.
- ▶ Any site in the world could read your grades if you're logged into Axess in another tab!

▶ لا! سيكون انتهاكًا كبيرًا لسياسة المنشأ نفسها.

▶ يمكن لأي موقع في العالم قراءة درجاتك إذا قمت بتسجيل الدخول إلى المحور في علامة تبويب أخرى!

Same origin or not?

نفس الأصل أم لا؟

- ▶ `https://example.com/a/ → https://example.com/b/`
 - ▶ Yes!
- ▶ `https://example.com/a/ → https://www.example.com/b/`
 - ▶ No! Hostname mismatch!
- ▶ `https://example.com/ → http://example.com/`
 - ▶ No! Protocol mismatch!
- ▶ `https://example.com/ → https://example.com:444/`
 - ▶ No! Port mismatch!
- ▶ `https://example.com/ → https://example.com:443/`
 - ▶ Yes!

Problems

مشاكل

- ▶ Sometimes policy is too **narrow**: Difficult to get **login.stanford.edu** and **axess.stanford.edu** to exchange data.
- ▶ Sometimes policy is too **broad**: No way to isolate **https://web.stanford.edu/class/cs106a/** from **https://web.stanford.edu/class/cs253/** ...much to CS 106A staff's disappointment! 😊
- ▶ Policy is not enforced for certain web features!

- ▶ You need to know which ones!

▶ في بعض الأحيان تكون السياسة ضيقة جدًا: من الصعب الحصول على **login.stanford.edu** و **axess.stanford.edu** لتبادل البيانات.

▶ في بعض الأحيان تكون السياسة واسعة جدًا: لا توجد طريقة لعزل **https://web.stanford.edu/class/cs106a/** من **https://web.stanford.edu/class/cs253/** ... الكثير مما تسبب في خيبة أمل موظفي CS 106A!

▶ لا يتم فرض السياسة على بعض ميزات الويب!

▶ عليك أن تعرف أي منها !

document.domain

- ▶ Idea: Need a way around Same Origin Policy to allow two different origins to communicate
- ▶ Two cooperating sites can agree that for the purpose of Same Origin Policy checks, they want to be considered equivalent.
- ▶ Sites must share a common top-level domain.
- ▶ Example: both **login.stanford.edu** and **axess.stanford.edu** may perform the following assignment:

`document.domain = 'stanford.edu'`

- ▶ الفكرة: بحاجة إلى طريقة للتغلب على نفس سياسة الأصل للسماح لأصلين مختلفين بالتواصل
- ▶ يمكن أن يتفق موقعان متعاونان على أنه لغرض عمليات التحقق من نفس سياسة الأصل، فإنهما يريدان اعتبارهما متكافئين.
- ▶ يجب أن تشترك المواقع في نطاق المستوى الأعلى المشترك.
- ▶ مثال: يمكن لكل من **login.stanford.edu** و **axess.stanford.edu** تنفيذ المهمة التالية :

Any subdomain can join the
document.domain party!

يمكن لأي نطاق فرعي الانضمام إلى
document.domain مجموعة

- ▶ So, if **axess.stanford.edu** and **attacker.stanford.edu** both run:

```
document.domain = 'stanford.edu'
```

- ▶ Then **attacker.stanford.edu** can access content on **axess.stanford.edu**!
Bad!

Demo

```
//From https://crypto.stanford.edu:  
document.domain = 'stanford.edu'
```

```
//From https://web.stanford.edu/class/cs253/:  
document.domain = 'stanford.edu'
```

```
// Is this allowed?  
iframe.contentDocument.body.style.backgroundColor = 'red' // Allowed
```

document.domain requires opt-in

- ▶ Both origins must explicitly opt-in to this feature
- ▶ So, if **attacker.stanford.edu** runs:

```
document.domain = 'stanford.edu'
```

- ▶ Then **attacker.stanford.edu** can NOT access content on stanford.edu!
- ▶ **stanford.edu** would also need to run the same code to opt-in to this behavior:

```
document.domain = 'stanford.edu'
```

- ▶ The above is not a no-op, despite how it looks!

- ▶ يجب على كلا المصدرين الاشتراك بشكل صريح في هذه الميزة
- ▶ لذلك، إذا كان **Attacker.stanford.edu** يعمل:

- ▶ ثم لا يمكن لـ **Attacker.stanford.edu** الوصول إلى المحتوى الموجود على stanford.edu!
- ▶ سيحتاج **stanford.edu** أيضًا إلى تشغيل نفس الكود للاشتراك في هذا السلوك:

- ▶ ما ورد أعلاه ليس محظورًا، على الرغم من مظهره!

Quiz

Originating URL	document.domain	Accessed URL	document.domain	Allowed?
http://www.example.com/	example.com	http://payments.example.com/	example.com	
http://www.example.com/	example.com	https://payments.example.com/	example.com	
http://payments.example.com/	example.com	http://example.com/	(not set)	
http://www.example.com/	(not set)	http://www.example.com/	example.com	

document.domain is a bad idea

- ▶ In order for login.stanford.edu and axess.stanford.edu to communicate, they must set:

```
document.domain = 'stanford.edu'
```

- ▶ This allows anyone on **stanford.edu** to join the party
 - ▶ Example: **attacker.stanford.edu** can also set **document.domain** to **stanford.edu** to become same origin with the others

لكي يتمكن تسجيل الدخول stanford.edu و axess.stanford.edu من التواصل، يجب عليهما تعيين:

```
document.domain = 'stanford.edu'
```

يُتيح هذا لأي شخص في stanford.edu الانضمام إلى الحفلة

▶ مثال: يمكن لـ مهاجم stanford.edu أيضاً تعيين document.domain إلى stanford.edu ليصبح نفس الأصل مع الآخرين

Send messages from a parent page to a child iframe

- ▶ Idea: Need a way around Same Origin Policy to allow two different origins to communicate
- ▶ What if we encoded data in URL fragment identifiers?
 - ▶ Gap in same origin policy!
 - ▶ Parent is allowed to navigate child iframes
 - ▶ Child can poll for changes to the fragment identifier

▶ الفكرة: بحاجة إلى طريقة للتغلب على نفس سياسة الأصل للسماح لأصلين مختلفين بالتواصل

▶ ماذا لو قمنا بتشفير البيانات في معرفات جزء URL؟

▶ فجوة في نفس سياسة المنشأ!

▶ يُسمح للوالدين بالتنقل في إطارات `iframe` التابعة

▶ يمكن للطفل إجراء استطلاع للتغييرات التي تم إجراؤها على معرف الجزء

Demo: Fragment identifier crossorigin communication

العرض التوضيحي: معرف الجزء عبر الاتصال عبر الأصل

```
<h1>Parent: http://site-a.com:8000</h1>
<input name='val' />
<br /><br />
<iframe src='http://site-b.com:8001' width='100%'
height='400px'></iframe>
<script>
  const input = document.querySelector('input')
  const iframe = document.querySelector('iframe')
  input.addEventListener('input', () => {
    iframe.src = `http://site-
b.com:8001#${encodeURIComponent(input.value)}`
  })
</script>
```

```
<h1>Child: http://site-b.com:8001</h1>
<div></div>
<script>
  const div = document.querySelector('div')
  setInterval(() => {
    div.textContent =
decodeURIComponent(window.location.hash).slice(1)
  }, 100)
</script>
```

The postMessage API

▶ Message

- ▶ The data to be sent to the other window. It can be a string, an object, or any other JSON-serializable data.

▶ targetOrigin

- ▶ Specifies the target window's origin. To enhance security, this parameter should be set to the specific origin of the target window (e.g., "https://example.com"). You can also use "*" to allow communication with any origin, but this is less secure.

▶ transfer (Optional):

- ▶ An array of Transferable objects (e.g., ArrayBuffer, MessagePort) that should be transferred instead of cloned when sent to the other window. This parameter is useful for improving performance when dealing with large data.

▶ رسالة

- ▶ البيانات المراد إرسالها إلى النافذة الأخرى. يمكن أن تكون سلسلة أو كائنًا أو أي بيانات JSON أخرى قابلة للتسلسل.

▶ targetOrigin

- ▶ يحدد أصل النافذة المستهدفة. لتعزيز الأمان، يجب تعيين هذه المعلمة على الأصل المحدد للنافذة المستهدفة (على سبيل المثال، "https://example.com"). يمكنك أيضًا استخدام "*" للسماح بالاتصال بأي أصل، لكن هذا أقل أمانًا.

▶ النقل (اختياري):

- ▶ مصفوفة من الكائنات القابلة للتحويل (على سبيل المثال، MessagePort ، ArrayBuffer التي يجب نقلها بدلاً من استنساخها عند إرسالها إلى النافذة الأخرى. هذه المعلمة مفيدة لتحسين الأداء عند التعامل مع البيانات الكبيرة.

`otherWindow.postMessage(message, targetOrigin, [transfer]);`

The postMessage API

- ▶ Secure cross-origin communications between cooperating origins
- ▶ Send strings and arbitrarily complicated data cross-origin
- ▶ Useful features:
 - ▶ "Structured clone" algorithm used for complicated objects. Handles cycles. Can't handle object instances, functions, DOM nodes.
- ▶ "Transferable objects" allows transferring ownership of an object. It becomes unusable (neutered) in the context it was sent from.

▶ تأمين الاتصالات عبر الأصل بين الأصول المتعاونة

▶ إرسال سلاسل وبيانات معقدة بشكل تعسفي عبر الأصل

▶ ميزات مفيدة:

▶ خوارزمية "الاستنساخ المنظم" المستخدمة للكائنات المعقدة. يتعامل مع الدورات. لا يمكن التعامل مع مثيلات الكائن والوظائف وعقد DOM.

▶ تسمح "الكائنات القابلة للتحويل" بنقل ملكية الكائن. يصبح غير صالح للاستخدام (مخصص) (في السياق الذي تم إرساله منه).

```
otherWindow.postMessage(message, targetOrigin, [transfer]);
```

Demo: postMessage cross-origin communication

```
<h1>Parent: http://site-a.com:8000</h1>
<input name='val' />
<br /><br />
<iframe src='http://site-b.com:8001' width='100%'
height='400px'></iframe>
<script>
  const input = document.querySelector('input')
  const iframe = document.querySelector('iframe')
  input.addEventListener('input', () => {
    iframe.contentWindow.postMessage(input.value
, 'http://site-b.com:8001')
  })
</script>
```

```
<h1>Child: http://site-b.com:8001</h1>
<div></div>
<script>
  const div = document.querySelector('div')
  window.addEventListener('message', event => {
    if (event.origin !== 'http://site-a.com:8000')
return
    div.textContent = event.data
  })
</script>
```

More realistic example

مثال أكثر واقعية

- ▶ **axess.stanford.edu** wants to display name of logged in user, so it registers a listener for messages:

```
window.addEventListener('message', event => {  
    setCurrentUser(event.data.name)  
}))
```

- ▶ Then it embeds an iframe to **login.stanford.edu** which runs:

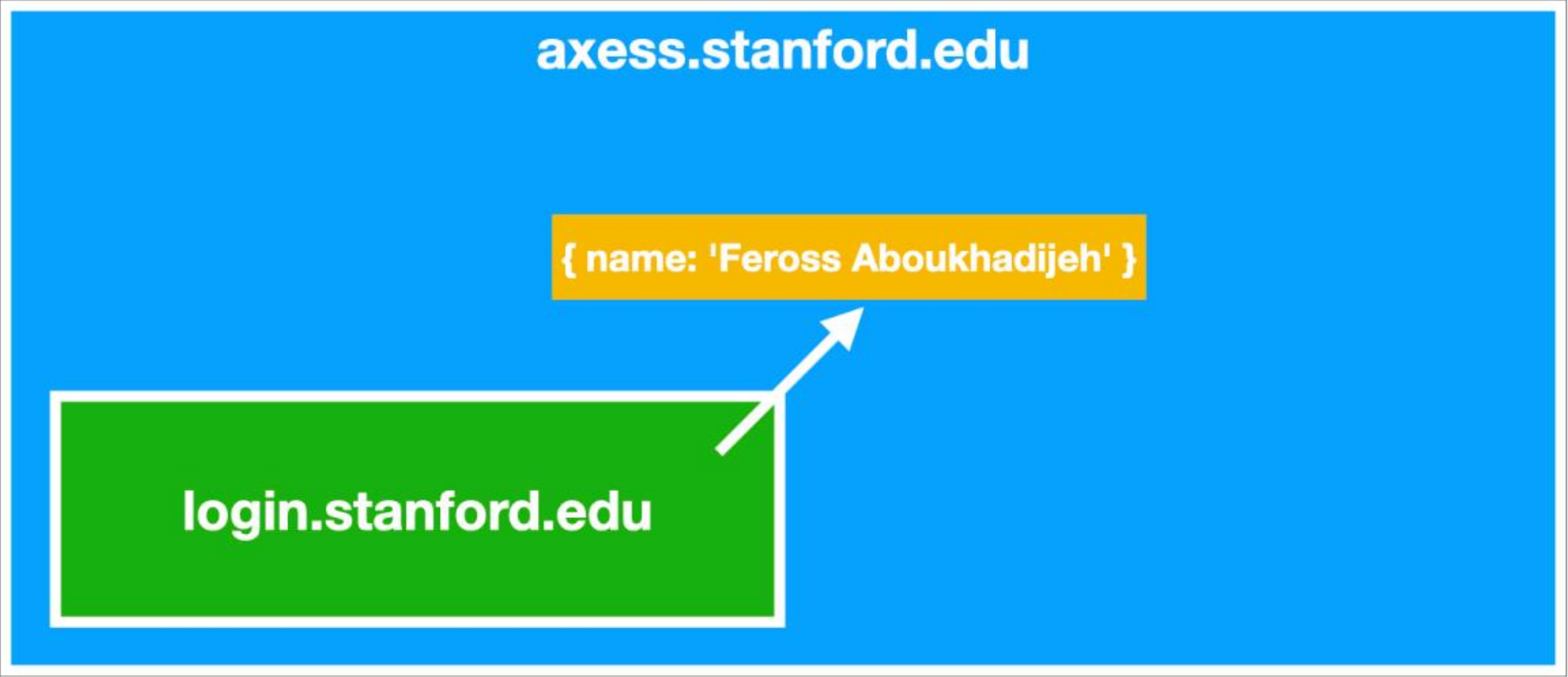
```
const data = { name: 'Feross Aboukhadijeh' }  
window.parent.postMessage(data, '*')
```

- ▶ This is insecure! Why?

axess.stanford.edu

{ name: 'Feross Aboukhadijeh' }

login.stanford.edu



attacker.com

{ name: 'Feross Aboukhadijeh' }

login.stanford.edu

```
graph LR; A[login.stanford.edu] --> B["{ name: 'Feross Aboukhadijeh' }"]; B --> C[attacker.com];
```

The diagram illustrates a data flow from a login page to an attacker's server. A green box labeled 'login.stanford.edu' has a white arrow pointing to a yellow box containing the JSON object '{ name: 'Feross Aboukhadijeh' }'. This yellow box is positioned between the login page and a large red box labeled 'attacker.com', indicating that the data is being sent to the attacker's server.

Need to validate destination of messages!

بحاجة إلى التحقق من صحة وجهة رسائل

- ▶ If an attacker embeds **login.stanford.edu**, they can listen to it's message which reveals the name of the logged in user!
- ▶ Solution: **login.stanford.edu** should specify intended recipient origin. Browser will enforce this.

▶ إذا قام أحد المهاجمين بتضمين **login.stanford.edu**، فيمكنه الاستماع إلى رسالته التي تكشف اسم المستخدم الذي قام بتسجيل الدخول!

▶ الحل: **login.stanford.edu** يجب أن يحدد أصل المستلم المقصود. سوف يفرض المتصفح هذا.

```
const data = { name: 'Feross Aboukhadijeh' }  
window.parent.postMessage(data, 'https://axess.stanford.edu')
```

attacker.com



axess.stanford.edu

{ name: 'Marc Tessier-Lavigne' }

login.stanford.edu

attacker.com

axess.stanford.edu

{ name: 'Marc Tessier-Lavigne' }

{ name: 'Feross Aboukhadijeh' }

login.stanford.edu



attacker.com

axess.stanford.edu

{ name: 'Marc Tessier-Lavigne' }

{ name: 'Feroz Boukhadijeh' }

login.stanford.edu



Need to validate source of messages!

بحاجة إلى التحقق من صحة مصدر الرسائل!

- ▶ If an attacker has a reference to a **axess.stanford.edu** window (by e.g. embedding it in an iframe), they can send a message to it to trick it!
- ▶ Solution: **axess.stanford.edu** should verify source origin of message!

```
window.addEventListener('message', event => {  
  if (event.origin !== 'https://login.stanford.edu') return  
  setCurrentUser(event.data.name)  
})
```

▶ إذا كان لدى المهاجم إشارة إلى نافذة **axess.stanford.edu** (على سبيل المثال، عن طريق تضمينها في **iframe**)، فيمكنه إرسال رسالة إليها لخداعها!

▶ الحل: يجب على **axess.stanford.edu** التحقق من أصل مصدر الرسالة!

Integrity of postMessage

- ▶ **Sender** must specify origin which is permitted to receive message
 - ▶ In case the URL of the target window has changed
- ▶ **Recipient** must validate the identity of the sender
 - ▶ In case some other window is sending the message
- ▶ **Remember:** Always specify intended recipient or expected sender!

▶ يجب على المرسل تحديد المصدر المسموح له بتلقي الرسالة

▶ في حالة تغيير عنوان URL للنافذة المستهدفة

▶ يجب على المستلم التحقق من صحة هوية المرسل

▶ في حالة قيام نافذة أخرى بإرسال الرسالة

▶ تذكر: حدد دائمًا المستلم المقصود أو المرسل المتوقع!

Same origin policy exceptions

نفس استثناءات سياسة الأصل

- ▶ **Summary:** There are **explicit opt-out** mechanisms like **document.domain**, fragment identifier communication, and the **postMessage** API
- ▶ There are also **automatic exceptions**
 - ▶ Need to be aware of these!
 - ▶ Source of many security issues!

▶ ملخص: هناك آليات واضحة لإلغاء الاشتراك مثل **document.domain**، واتصالات معرف الأجزاء، و**postMessage API**

▶ هناك أيضًا استثناءات تلقائية

▶ يجب أن تكون على علم بهذه!

▶ مصدر العديد من القضايا الأمنية !

Same origin policy - automatic exceptions

نفس سياسة الأصل - الاستثناءات التلقائية

- Which of these requests from example.com are allowed?

All of them

```
<!doctype html>
<html lang='en'>
<head>
  <meta charset='utf-8' />
  <link rel='stylesheet' href='https://other1.com/style.css' />
</head>
<body>
  <img src='https://other2.com/image.png' />
  <script src='https://other3.com/script.js'></script>
</body>
</html>
```

Same origin policy exceptions

نفس استثناءات سياسة الأصل

- ▶ Embedded static resources can come from another origin
 - ▶ Images (e.g. hotlinking to memes)
 - ▶ Scripts (e.g. Facebook like button, ads, tracking scripts)
 - ▶ Styles (e.g. Google Fonts)
- ▶ **Why was it designed this way?**

▶ يمكن أن تأتي الموارد الثابتة المضمنة من أصل آخر

▶ الصور (مثل الارتباط السريع بالميقات)

▶ البرامج النصية (مثل زر الإعجاب على Facebook، والإعلانات، ونصوص التتبع

▶ (الأنماط) مثل خطوط (Google)

▶ لماذا تم تصميمه بهذه الطريقة؟